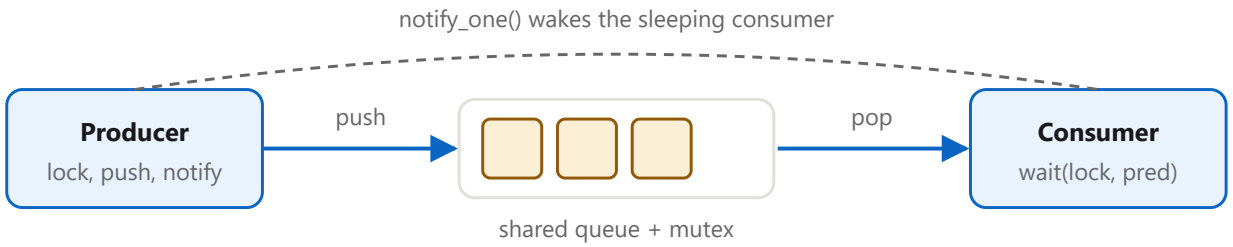
Condition Variables and Debugging

How a thread waits for an event without burning a CPU, the three rules, a reusable thread-safe queue, and the tools and symptoms for finding thread bugs. Assumes Deadlock.

Condition variables



The consumer sleeps (no CPU) until notified, then re-checks the predicate to survive spurious wakeups.

Condition variable: sleep until something changed, always paired with a mutex and a condition.

To **wait until something happens**, a busy loop burns 100% CPU. A **condition variable** lets the thread *sleep* until woken, always together with a mutex and the shared state.

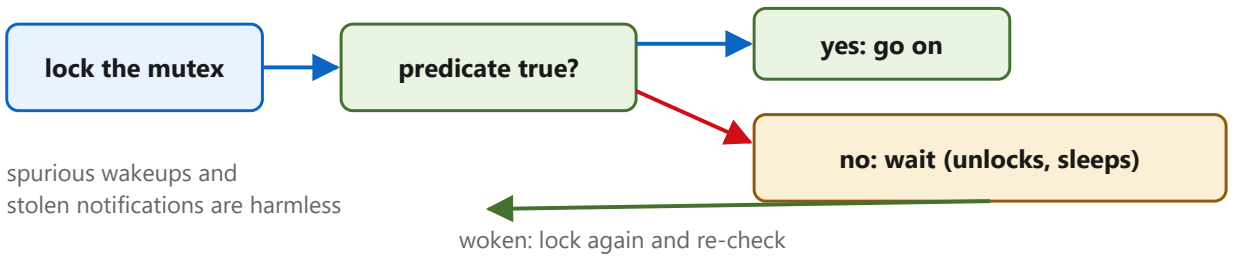
```
std::mutex m;
std::condition_variable cv;
std::queue<int> q;

// CONSUMER
std::unique_lock<std::mutex> lk(m);
// sleep until the predicate is true
cv.wait(lk, [] { return !q.empty(); });
int x = q.front(); q.pop();

// PRODUCER
{ std::lock_guard<std::mutex> lk(m); q.push(42); }
cv.notify_one();                              // wake one waiting thread
```

The predicate is the truth

The two ideas that make condition variables safe are worth one picture. A thread never trusts a wakeup: it re-checks the condition, under the lock, every time it wakes.



The predicate is the truth; the notification is only a hint to look again.

MISTAKE	WHAT GOES WRONG	FIX
waiting without a predicate	a spurious wakeup or a notification that came before the wait is missed or misread	`cv.wait(lk, pred)`
changing the condition without the mutex	the waiter checks, the notifier changes and notifies, the waiter then sleeps forever	modify the shared state under the same mutex
`notify_one` when several threads wait for different things	the wrong thread wakes and goes back to sleep	`notify_all`, or separate condition variables

Three rules to follow

1. **Always wait with a predicate.** Spurious wakeups happen, and another consumer may take the item first.
2. **Change the state under the mutex**, then notify. Otherwise a lost wakeup: the waiter sees false, you notify, it then sleeps forever.
3. The waiter needs `unique_lock`: `wait` unlocks while sleeping.

`notify_one()` wakes one waiter; `notify_all()` wakes all (shutdown, or several may proceed).

A reusable thread-safe queue

```
template <typename T>
class SafeQueue {
    std::queue<T> q_;
    std::mutex m_;
    std::condition_variable cv_;
public:
    void push(T v) {
        { std::lock_guard<std::mutex> lk(m_); q_.push(std::move(v)); }
        // notify AFTER releasing the lock: the woken thread can take it
        // at once
        cv_.notify_one();
    }
    // blocks until an item exists
    T pop() {
        std::unique_lock<std::mutex> lk(m_);
        cv_.wait(lk, [this] { return !q_.empty(); });
        T v = std::move(q_.front());
        q_.pop();
        return v;
    }
};
```

Debugging and common bugs

Timing bugs vanish under a debugger or a `printf` ("Heisenbug"). Use tools:

- **ThreadSanitizer**: `-fsanitize=thread -g` finds data races and prints both accesses. Run tests with it.
- **Helgrind / DRD** (Valgrind): races and lock-order violations.
- Thread ids in logs; `gdb` `thread apply all bt` on a hang.

SYMPTOM	LIKELY CAUSE
Wrong totals, rare crashes	data race
Hang, CPU idle	deadlock or lost wakeup
Hang, CPU 100%	busy-wait or livelock
Crash at exit	detached thread using destroyed data; joinable thread destroyed
Slower with more threads	lock contention, false sharing, oversubscription
Garbled output	several threads on `std::cout`: build the line, print once, or lock

---

Vocabulary

- **Thread-safe**: usable from several threads without external locking (`shared_ptr`'s reference count, *not* its pointee).
- `thread_local`: one copy per thread.
- **Function-local** `static` init is thread-safe since C++11 (Meyers Singleton).
- `std::call_once(flag, fn)`: initialize exactly once.

> WHERE NEXT Atomics and the memory model are in Atomics and the Memory Model, futures, thread pools and C++20 coordination tools in Tasks and Parallelism.

Further reading: your notes in Module 3: multithreading and synchronization (with runnable examples), the modern-cpp-tutorial concurrency chapter (threads, mutexes, futures, condition variables, atomics and the memory model), and the modern-cpp-features notes on `std::thread`, `std::async`, `std::scoped_lock` and parallel algorithms.

A producer and consumer timeline

The whole pattern is easiest to see on a timeline: the consumer sleeps without using the CPU, the producer pushes and notifies, the consumer wakes, takes the item and sleeps again.



Each wake-up re-checks !q.empty() under the lock; an empty queue means back to sleep (a spurious wakeup is harmles

Between wake-ups the consumer uses no CPU.

SITUATION	WHAT HAPPENS
notify arrives before the consumer waits	the consumer checks the predicate first, sees the item and never sleeps: no lost wakeup
two items pushed, one notify	the consumer pops one, re-checks, finds another and pops that too: a loop around wait handles it
notify with nobody waiting	nothing: a condition variable does not remember notifications, the shared state does
shutdown	set a stop flag under the lock and `notify_all`: every waiter wakes, sees the flag and returns

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Condition variables	Effective Modern C++	Item 39, Consider void futures for one-shot event communication	280 to 289
	C++ Concurrency in Action	4.1 Waiting for an event: condition variables and a thread-safe queue	96 to 104
	C++ Concurrency in Action	6.1 to 6.2.2 Designing for concurrency, a thread-safe stack and a queue with condition variables	197 to 206
Debugging and common bugs	C++ Concurrency in Action	11.1 Types of concurrency-related bugs, and 11.2.1 reviewing code to locate them	363 to 367